

1. Feltyper

Syntaxfel

När koden bryter mot språkets regler (GeeksforGeeks, 2025). Exempel:

```
def funktion(x) return x ** 2
```

funktionen ovan saknas kolon efter

I funktionen ovan saknas kolon efter parentes, vilket gör att programmet kraschar.

Felmeddelandet är `SyntaxError` och visar användaren vad som är fel samt på vilken rad felet ligger.

Logiska fel

När koden är rätt skriven, men gör fel sak (GeeksforGeeks, 2025). Exempel:

```
total = 0
for i in range(1, 6):
    total = 0
    total += i
print(total)
```

Här blir output 5, medan önskat output är 15. Felet är att 'total' återställs till 0 efter varje iteration, så att i den sista iterationen är 'total += i' lika med '0 += 5' vilket ger 5. Rätt skriven kod hade endast haft 'total = 0' utanför loopen.

Runtime-fel

När programmet kraschar under körning (GeeksforGeeks, 2025). Ett exempel är `ZeroDivisionError`, som innebär att programmet kraschar när användaren försöker dividera ett tal med 0. Ett annat exempel är `TypeError`:

```
x = 5
y = " timmar"
print(x + y)
```

Här kraschar programmet eftersom man försöker addera ett heltal med en sträng. Ett sätt att komma runt detta är att byta ut plustecknet mot ett kommatecken.

2. Undantagshantering med try/except

```
while True:

    try:
        tal = int(input("Ange ett tal: "))
        print(f"100 / {tal} = {100 / tal:.2f}")
        break

    except ValueError:
        print("Det måste vara ett heltal!")

    except ZeroDivisionError:
        print("Kan inte dela med noll!")
```

För att köra kod som kan ge upphov till ett fel används 'try' (W3Schools.com). I exemplet ovan frågas användaren efter ett heltal, men om användaren exempelvis anger ett decimaltal eller en sträng kommer koden inte att kompilera. Om Python hittar ett fel används 'except' för att hantera det (W3Schools.com). Man kan till exempel ange ett felmeddelande som ska visas om användaren har matat in ett felaktigt input, vilket orsakar ett 'ValueError'. I exemplet har jag använt både 'ValueError' och 'ZeroDivisionError' (om Python upptäcker division med noll) som möjliga undantag. Man kan säga att 'except' hindrar programmet från att krascha om ett fel upptäcks, och istället kan man hantera felet som man själv vill.

3. Pseudokod

START

UPPREPA

FRÅGA användaren om deras ålder
LÄS in ålder

OM ålder inte är ett giltigt heltal:
SKRIV UT felmeddelande
BÖRJA OM

ANNARS om ålder är 18 eller äldre
SKRIV UT "Du är vuxen"
AVSLUTA LOOPEN

ANNARS om ålder är mellan 13 och 17
SKRIV UT "Du är tonåring"
AVSLUTA LOOPEN

ANNARS om ålder är under 13
SKRIV UT "Du är barn"
AVSLUTA LOOPEN

SLUT

4. Reflektion

Det är praktiskt att skriva pseudokod innan man skriver riktig kod eftersom man får en tydlig översikt av hur det färdiga programmet kommer att fungera. Utan planering blir det enligt mig lätt att råka överkomplicera programmet eller att upprepa kod i onödan. Att planera sitt program med hjälp av pseudokod minskar därför antalet fel, eftersom man ser precis vilka delar programmet ska innehålla samt ramarna för hur/när det ska avslutas, etc.

I den här uppgiften har jag lärt mig att hantera undantag med hjälp av try/except, att planera program med hjälp av pseudokod, samt fått en bättre förståelse för vilka olika typer av fel som kan uppstå; såsom syntaxfel, logiska fel och runtime-fel. Logiska fel är enligt mig svårast att hantera, eftersom de inte ger upphov till något felmeddelande.

Källförteckning

GeeksforGeeks (2025-07-23). *Types of Issues and Errors in Programming/Coding*.

<https://www.geeksforgeeks.org/dsa/types-of-issues-and-errors-in-programming-coding/#3-runtime-errors> [2026-07-16]

W3Schools.com (u.å.). *Python Try Except*.

https://www.w3schools.com/python/python_try_except.asp [2026-07-16]